

Technical Research Report: Real-Time Pitch Detection and Visualization Architecture for the Web

1. Executive Summary and Architectural Vision

The migration of Digital Signal Processing (DSP) from native, compiled environments to the web browser represents a paradigm shift in how users interact with audio software. Historically, real-time pitch detection—the ability to instantaneously identify the fundamental frequency (f_0) of a musical signal—was the exclusive domain of C++ applications using frameworks like JUCE or low-level CoreAudio interfaces. The latency requirements for a "real-time" experience, generally perceived as under 20 milliseconds, posed insurmountable challenges for the single-threaded, garbage-collected nature of early JavaScript engines. However, the maturation of the W3C Web Audio API, specifically the introduction of the AudioWorklet interface and the widespread support for WebAssembly (Wasm), has fundamentally altered this landscape. It is now possible to build professional-grade audio analysis tools that run entirely within a standard web browser, without plugins. This technical spike report details the research, architectural decisions, and implementation strategies required to construct a real-time pitch detection and visualization web application, functionally equivalent to reference implementations like PitchReader.

The core challenge identified in this research is not merely algorithmic selection, but the architectural orchestration of three distinct processing domains: the Main Thread (UI/Visualization), the Audio Thread (DSP/Capture), and potentially a Worker Thread (Heavy Computation). A successful Minimum Viable Product (MVP) must navigate the inherent tension between the browser's asynchronous event loop and the synchronous, isochronous demands of digital audio.

This report concludes that a hybrid architecture utilizing the **McLeod Pitch Method (MPM)** running within an **AudioWorklet**, communicating via lightweight message passing to a hardware-accelerated **HTML5 Canvas** visualization layer, offers the optimal balance of latency, accuracy, and implementation complexity for the target MVP. While deep learning approaches like CREPE offer superior accuracy, their computational overhead renders them unsuitable for the strictly real-time constraints of this specific project on consumer hardware.

2. Audio Acquisition and the Web Audio API Landscape

The foundation of any pitch detection system is the acquisition of a high-fidelity digital representation of the acoustic signal. In the browser, this is managed through the Web Audio API, a high-level JavaScript API for processing and synthesizing audio. The research indicates

that while the API is powerful, it contains several "traps" for the unwary developer—specifically, deprecated nodes that remain in documentation but offer sub-par performance, and default configurations that actively interfere with musical analysis.

2.1 The Evolution of Browser Audio: From Tags to Graphs

To understand the recommended architecture, one must contextualize the evolution of audio on the web. Early implementations relied on the `<audio>` tag, which was limited to playback and offered no access to the raw PCM (Pulse Code Modulation) stream. The introduction of the Web Audio API changed this by modeling audio as a routing graph. In this graph, "nodes" (such as sources, gain controls, and analyzers) are connected to form a processing chain. The audio signal flows through these nodes in discrete blocks, typically 128 sample frames at a time.

The most critical realization for this project is that this graph execution happens separately from the JavaScript main thread. This separation is vital because the main thread is shared with the DOM, CSS rendering, and user interaction. If audio processing were bound to the main thread, scrolling the page or triggering a CSS animation would cause the audio stream to stutter or "glitch".¹

2.2 Accessing the Hardware: `getUserMedia` and Constraints

The entry point for capturing audio is the `MediaStream` interface, accessed via `navigator.mediaDevices.getUserMedia()`. While seemingly straightforward, this API applies heavy pre-processing by default, optimized for speech intelligibility in teleconferencing rather than musical fidelity.

2.2.1 The Destructive Nature of Default Constraints

When a browser requests microphone access, it typically enables **Echo Cancellation**, **Noise Suppression**, and **Auto Gain Control (AGC)**.

- **Echo Cancellation:** Attempts to remove audio playing from the speakers from the input signal. This involves spectral subtraction and phase manipulation.
- **Noise Suppression:** Targets steady-state background noise (like fans) but often interprets the sustained decay of a guitar string or a held vocal note as noise, aggressively gating or distorting the tail of the sound.
- **Auto Gain Control:** Dynamically adjusts input volume. For a pitch detector, this causes the signal-to-noise ratio to fluctuate wildly, complicating thresholding logic.

Research confirms that for pitch detection, these features must be explicitly disabled. The algorithm requires the rawest possible waveform to accurately analyze the periodicity of the signal. The recommended constraint configuration is:

JavaScript

```
const musicalConstraints = {
```

```

audio: {
  echoCancellation: false,
  autoGainControl: false,
  noiseSuppression: false,
  channelCount: 1, // Mono is sufficient for pitch detection
  latency: 0      // Requests the lowest possible latency mode
}
};

```

Failing to disable these constraints is a primary cause of detection instability in web-based tuners.²

2.3 Audio Processing Architectures: The Shift to AudioWorklet

Once the stream is captured, the raw data must be intercepted for analysis. This is where the most significant architectural divergence occurs between legacy tutorials and modern best practices.

2.3.1 The Deprecated ScriptProcessorNode

For years, the standard method for accessing raw audio was the ScriptProcessorNode. This node would buffer audio chunks and fire an onaudioprocess event on the **Main Thread**.

- **Mechanism:** The audio thread would pause, serialize the audio buffer, cross the thread boundary to the main thread, wait for the JS event loop to run the handler, and then deserialize the result.
- **Performance Implication:** This design introduced unavoidable latency and instability. If the main thread was busy (e.g., reflowing the DOM), the audio engine would run out of data, resulting in audio dropouts. For pitch detection, this meant that high CPU load (like drawing the visualization) could cause the detector to miss segments of audio entirely.³

2.3.2 The Modern Standard: AudioWorklet

The AudioWorklet interface was introduced specifically to resolve the thread-blocking issues of ScriptProcessorNode.

- **Mechanism:** It allows developers to supply a JavaScript (or WebAssembly) module that runs *inside* the Audio Thread.
- **Isolation:** The code executing in the AudioWorkletProcessor is isolated from the main thread. It has no access to the DOM or global variables.
- **Synchronicity:** The process() method of the worklet is called synchronously by the audio subsystem every 128 frames (approximately 2.9ms at 44.1kHz).
- **Benefits:** This ensures that audio capture and analysis are perfectly isochronous. Even if the visualization freezes completely, the pitch detector continues to process the incoming stream without dropping a single sample. This separation of concerns—Visualization on the Main Thread, Analysis on the Audio Thread—is the

cornerstone of a high-performance architecture.³

2.4 Browser Compatibility and Security Contexts

The Web Audio API is widely supported, but implementations vary.

- **Google Chrome / Edge:** Excellent support for AudioWorklet and high-performance threading.
- **Firefox:** Strong support, though latency tuning can be different on Linux.
- **Safari (WebKit):** Support for AudioWorklet was added in Safari 14.5. However, Safari enforces stricter autoplay policies. The AudioContext starts in a suspended state and must be explicitly resumed inside a user-initiated event handler (e.g., a "Start" button click). Failing to handle this state transition is a common point of failure on iOS devices.⁵
- **Secure Context:** Advanced features like AudioWorklet and high-precision timers often require the page to be served over HTTPS. Localhost is treated as secure for development, but deployment requires a valid SSL certificate.⁷

3. Pitch Detection Algorithms: Theoretical Analysis

Detecting pitch is distinct from detecting frequency. "Frequency" is a physical property of a periodic wave, while "Pitch" is a psychoacoustic percept. For a pure sine wave, they are identical. For complex musical tones—which consist of a fundamental frequency (f_0) plus a series of harmonic overtones ($2f_0, 3f_0, \dots$)—the challenge lies in correctly identifying the fundamental, even when it is weak or missing (the "missing fundamental" phenomenon).

This section evaluates the algorithmic options available for real-time browser implementation, weighing the trade-offs between computational cost (latency) and harmonic robustness (accuracy).

3.1 Time-Domain Algorithms

Time-domain algorithms operate directly on the amplitude values of the waveform. They generally offer better time resolution than frequency-domain methods, making them superior for low-latency real-time applications.

3.1.1 Zero-Crossing Rate (ZCR)

The simplest method involves counting how many times the signal crosses the zero-amplitude line.

- **Mechanism:** $\text{Frequency} = (\text{Sample Rate}) / (\text{Number of samples between crossings})$.
- **Limitations:** It fails catastrophically in the presence of harmonics or noise. A complex wave might cross zero multiple times within a single period due to strong overtones. It is unsuitable for musical pitch detection.⁸

3.1.2 Autocorrelation (ACF)

Autocorrelation is the mathematical basis for most modern pitch detectors. It measures the similarity of a signal with a time-shifted version of itself.

- **Formula:** $r_t(\tau) = \sum_{j=t+1}^{t+W} x_j x_{j+\tau}$
 - Where x is the signal, W is the window size, and τ is the lag (time shift).
- **Peak Picking:** When the lag τ matches the true period of the signal, the overlapping waveforms align constructively, producing a peak in the correlation function.
- **The Octave Error Problem:** A signal with a period T is also periodic at $2T$, $3T$, etc. Pure autocorrelation often identifies these sub-harmonics as the pitch (e.g., detecting 110Hz instead of 220Hz), or conversely, locks onto a strong harmonic. This makes raw ACF unreliable for general music usage.²

3.1.3 The YIN Algorithm

Published by De Cheveigné and Kawahara in 2002, YIN is widely regarded as the state-of-the-art for monophonic pitch detection. It is essentially a "smart" autocorrelation that addresses the octave error problem through a series of refinement steps.¹⁰

- **Step 1: Difference Function:** Instead of maximizing correlation (product), YIN minimizes the squared difference. This is computationally more stable.
- **Step 2: Cumulative Mean Normalization:** This is the key innovation. It normalizes the difference function by the average difference up to that lag. This prevents the algorithm from falsely selecting the zero-lag dip (which is always 0) and suppresses high-frequency errors.
- **Step 3: Absolute Thresholding:** Instead of picking the absolute global minimum (which might be an octave error), YIN picks the *first* local minimum that dips below a set threshold (e.g., 0.1). This biases the selection toward the shortest valid period (highest pitch), effectively solving the octave-too-low error.
- **Step 4: Parabolic Interpolation:** The lag τ is an integer (sample count). To detect pitch with precision (e.g., 440.0 Hz vs 440.5 Hz), YIN fits a parabola around the minimum to estimate the fractional lag.

Performance Profile: YIN is computationally heavier than simple ACF but runs comfortably in real-time on modern processors. It is robust against the "missing fundamental" and works well for both voice and instruments.

3.1.4 The McLeod Pitch Method (MPM)

MPM is a variation of autocorrelation that utilizes a Normalized Square Difference Function (NSDF).

- **Comparison to YIN:** While YIN minimizes difference, MPM maximizes normalized correlation.
- **Key Feature:** It normalizes the correlation by the signal energy, yielding values between -1 and 1. This makes thresholding intuitive (e.g., "accept peaks above 0.9").
- **Efficiency:** MPM is often cited as being slightly faster than YIN and particularly effective for string instruments (guitar/violin). It is the algorithm of choice for several popular JavaScript libraries like pitchy.¹¹

3.2 Frequency-Domain Algorithms

These methods use the Fast Fourier Transform (FFT) to convert the signal into a spectrum.

3.2.1 Harmonic Product Spectrum (HPS)

HPS exploits the harmonic series. It downsamples the spectrum by integer factors (2, 3, 4...) and multiplies them together. The harmonic peaks align, while random noise does not, causing the fundamental to emerge as the dominant peak.

- **Trade-offs:** HPS is computationally efficient but requires a large FFT window to achieve good frequency resolution at the low end (bass notes). This introduces latency. For a frequency resolution of 1 Hz at 44.1kHz, one needs a window of ~44,000 samples (1 second), which is unacceptable for real-time use.

3.3 Machine Learning Approaches

3.3.1 CREPE (Convolutional Representation for Pitch Estimation)

CREPE is a deep neural network trained directly on time-domain waveforms to predict pitch.

- **Accuracy:** It significantly outperforms signal-processing methods (YIN/MPM) in noisy environments (e.g., singing in a crowded room).¹⁴
- **Computational Cost:** Inference requires running a Convolutional Neural Network (CNN). In a browser, this relies on TensorFlow.js and WebGL acceleration. Even with optimization, the inference latency is typically higher than the 20ms target for a smooth visualization. It is better suited for offline transcription than live interaction on average consumer hardware.¹⁶

3.4 Comparative Summary

Algorithm	Method	Accuracy (Clean)	Accuracy (Noisy)	Latency	Computational Cost	Recommended For
Autocorrelation	Time	Low	Low	Very Low	Low	Educational Demos
YIN	Time	High	Medium	Low	Medium	Vocals/General
MPM	Time	High	Medium	Low	Medium	Instruments
HPS	Freq	Medium	Low	High	Low	Simple Tuners
CREPE	ML	Very High	High	High	Very High	Offline Analysis

Recommendation for MVP: The **McLeod Pitch Method (MPM)** or **YIN** represents the optimal balance. They are fast enough to run in an AudioWorklet (even in JavaScript) and accurate enough for a satisfying user experience.

4. Implementation Strategy: The Signal Pipeline

Building the application involves constructing a pipeline that moves data from the microphone to the screen. This section details the technical implementation of that pipeline.

4.1 Sample Rate and Buffering

The `AudioWorkletProcessor.process()` method receives 128 samples per call. At 44.1kHz, this is roughly 2.9ms of audio. However, pitch detection algorithms need a longer window to detect low frequencies.

- **Physics of Low Notes:** The low E on a guitar is ~82.4 Hz. One full cycle takes $1/82.4 \approx 12.1$ ms.
- **Window Requirement:** To detect periodicity accurately, algorithms like YIN need at least 2-3 full cycles of the wave. This implies a minimum window size of ~36ms.
- **Buffer Size:** A window of **2048 samples** at 44.1kHz provides ~46ms of audio, covering almost 4 cycles of low E. This is the ideal buffer size.

The Ring Buffer Implementation:

Since the worklet receives 128 samples but needs 2048 to run the algorithm, we must implement a Ring Buffer (Circular Buffer).

1. **Allocation:** A `Float32Array` of size 2048 is allocated at startup.
2. **Write Pointer:** An integer tracks the current write position.
3. **Accumulation:** Inside `process()`, the 128 input samples are copied into the ring buffer.
4. **Circular Wrap:** If the write pointer exceeds 2048, it wraps back to 0.
5. **Triggering:** We don't need to run the pitch detector every 128 samples (which would be 344 times a second—overkill). Instead, we can run it every 512 or 1024 samples (approx. 43 or 21 times a second). This "Hop Size" balances CPU load with update frequency.

4.2 Handling Thread Communication

Once the `AudioWorklet` detects a pitch, it must send the data to the Main Thread for visualization.

- **MessagePort:** The standard way is `this.port.postMessage({ pitch: 440.0, clarity: 0.98 })`.
- **Garbage Collection Warning:** Creating a new object `{...}` 40 times a second creates garbage. In high-performance scenarios, it is better to reuse a pre-allocated object or use a `SharedArrayBuffer` backed by `Atomics` for lock-free communication.
 - *MVP approach:* For an MVP, `postMessage` is acceptable, but the message payload should be kept minimal.

4.3 Signal Conditioning and Post-Processing

Raw pitch detection data is often "jittery." A note held at 440 Hz might fluctuate: 439, 441, 440, 220 (octave error), 440. To make the graph look good, post-processing is required.

1. **Confidence Thresholding:** Ignore any result where the algorithm's "clarity" or "probability" metric is below a certain threshold (e.g., 0.9). This acts as a Noise Gate,

preventing the graph from drawing random scribbles when the room is silent.¹²

2. **Median Filtering:** To remove transient octave errors (single-frame dropouts), apply a Median Filter of size 3 or 5 to the pitch stream. Unlike a mean (average) filter, a median filter cleanly rejects outliers without "smearing" the transition between notes.¹⁸
3. **Hz to Note Conversion:** The visualization likely needs to display note names (C4, A#). The formula is:

$$n = 12 \times \log_2 \left(\frac{f}{440} \right) + 69$$

where n is the MIDI note number.

5. Visualization Technology Stack

The requirement is a real-time scrolling graph (often called a "waterfall" or "ticker tape" display). This presents specific rendering challenges.

5.1 Technology Comparison: DOM vs. SVG vs. Canvas

- **DOM Elements (HTML divs):** Creating a div for every data point is extremely slow. Modifying the DOM triggers Layout and Paint operations. This approach effectively caps at <100 points before performance degrades.
- **SVG (Scalable Vector Graphics):** SVG is better, using a single <polyline>. However, updating the points attribute of a path with thousands of coordinates 60 times a second can still bottleneck the browser's style recalculation engine.¹⁹
- **HTML5 Canvas (2D Context):** This is the optimal choice for the MVP. It operates in "immediate mode"—pixels are drawn and then forgotten. It incurs no overhead for the number of objects drawn, only for the number of drawing commands issued.
- **WebGL:** While faster for millions of points, the complexity of writing shaders and managing vertex buffers is unnecessary for a simple 2D line graph. Canvas 2D is hardware accelerated on modern browsers and sufficient for this use case.²¹

5.2 Optimizing the Scrolling Effect

Naively clearing and redrawing the entire graph every frame is inefficient ($O(N)$ operation). Two optimization techniques are relevant:

5.2.1 The drawImage Shift Technique

This method utilizes the canvas itself as a buffer.²³

1. **Draw:** `ctx.drawImage(canvas, -dx, 0)` draws the *current* canvas image back onto itself, shifted to the left by dx pixels.
2. **Clear:** `ctx.clearRect(width - dx, 0, dx, height)` clears the dirty strip on the right edge.
3. **Plot:** Draw the new data point in the cleared strip.
4. **Benefit:** The cost is $O(1)$ —it is constant regardless of how wide the screen is or how much history is shown.

5.2.2 The Circular Buffer Render

Store the last N pitch values in a JavaScript array.

1. **Pointer:** Keep a generic circular buffer index.
2. **Clear:** `ctx.clearRect(0, 0, width, height)`.
3. **Path:** Begin a path, iterate through the circular buffer starting from the "oldest" data, mapping index to X coordinate.
4. **Draw:** `ctx.stroke()`.
5. **Benefit:** While technically $O(N)$, for $N < 2000$ (width of a typical screen), modern engines can stroke a path almost instantly. This produces cleaner anti-aliasing than the shift technique, which can blur lines over time due to repeated sub-pixel resampling.²⁵

Recommendation: The **Circular Buffer Render** is preferred for visual crispness, as modern devices can easily handle redrawing 2000 line segments at 60fps.

6. Implementation Challenges and Solutions

6.1 The "Jank" Problem (Garbage Collection)

In JavaScript, creating objects triggers memory allocation. When memory fills, the Garbage Collector (GC) runs, pausing execution. In a 60fps visualization, a 20ms GC pause causes a visible "stutter."

- **Solution: Object Pooling.** Never use `new` inside the render loop or the audio process loop. Pre-allocate all arrays, objects, and vectors at application startup. Reuse them frame-to-frame.

6.2 Latency vs. Smoothing

There is an inherent conflict between responsiveness and stability. Heavy smoothing makes the pitch graph look clean but feel "laggy" (like a rubber band).

- **Solution:** Use a **variable smoothing factor**. When the pitch changes drastically (a new note attack), reduce smoothing to jump quickly. When the pitch is stable (sustained note), increase smoothing to reduce jitter. This "Adaptive Filtering" provides the best of both worlds.

6.3 Mobile Browser Quirks

- **Sample Rate Mismatch:** Mobile hardware often runs at 48kHz or 44.1kHz, but sometimes weird rates like 44.1kHz on input and 48kHz on output. The math must use `audioContext.sampleRate` dynamically, never hardcoded constants.
- **Power Saving:** If the user switches tabs, `requestAnimationFrame` stops, but `AudioWorklet` might continue (or be throttled). The application logic must handle the "catch-up" when the tab becomes active again, or explicitly pause processing to save battery.

7. Recommended Technology Stack for MVP

Based on the research, the following stack is recommended:

7.1 Core Dependencies

- **Language: TypeScript.** Strong typing is invaluable for managing the complex data structures (buffers, audio params) and ensuring thread safety contracts.
- **Audio Library: Pitchy** (npm). It is a robust, zero-dependency implementation of MPM. It is lightweight enough to be bundled directly into the AudioWorklet processor file without complex build steps.¹¹
 - *Alternative:* If performance on low-end Android devices proves problematic, switch to **AubioJS** (WASM), but this adds build complexity.
- **Build Tool: Vite.** It has excellent support for AudioWorklet and Worker bundling out of the box, simplifying the "module loading" pain points often found with Webpack.

7.2 Frontend

- **Framework: React** or **Vue** (lightweight). Use the framework *only* for the UI shell (buttons, settings menus).
- **Visualization: Vanilla Canvas API.** Encapsulate the canvas logic in a `useLayoutEffect` (React) or `onMounted` (Vue) hook. Do *not* let the framework's reactivity system manage the canvas updates.

7.3 Data Flow Architecture

1. **Main Thread:** AudioContext setup, UI, Visualization Loop.
2. **Audio Thread:** AudioWorklet running pitchy.
3. **Communication:** `port.postMessage` sending float (frequency) and float (clarity) at ~30-60Hz.

8. Existing Solutions and Comparison

Several open-source projects provide reference points, though none offer a complete "plug-and-play" solution for the specific requirements of a scrolling visualization.

Solution	Technology	Pros	Cons
PitchDetect (Chris Wilson)	ScriptProcessorNode / ACF	Simple code, easy to read	Deprecated API, high latency, main-thread blocking ²⁶
Pitchy (Library)	MPM / Pure JS	Modern, accurate, lightweight	Library only, no visualization included ¹¹
AubioJS	WASM / YIN	Extremely fast, industry standard	Complex build, large binary size ²⁷
PitchReader (Target)	Likely Custom	Slick UI, smooth graphing	Closed source, reference only

9. Conclusion

The "Deep Research" indicates that building a real-time pitch detection app is a solved problem algorithmically, but a nuanced problem architecturally. The key to success is not inventing a new way to detect pitch, but strictly adhering to the **AudioWorklet** pattern to ensure the audio processing is immune to UI jank.

By utilizing the **McLeod Pitch Method** for its balance of speed and accuracy, and rendering the output via **Hardware-Accelerated Canvas**, the proposed application can achieve the sub-20ms latency required for a responsive musical experience. This report provides the blueprint for that implementation, detailing the specific traps (like default microphone constraints and garbage collection pauses) that must be avoided to move from a "tech demo" to a robust product.

10. References and Citations

Data points and technical claims in this report are supported by the following research snippets:

- **Web Audio API & Worklets:**¹
- **Pitch Detection Algorithms:**²
- **Canvas & Visualization:**²³
- **Browser Constraints:**²
- **Existing Libraries:**¹¹

11. Detailed Technical Specifications: Audio Processing Pipeline

This section expands on the implementation details of the audio pipeline, specifically focusing on the critical interaction between the main thread and the AudioWorklet. This relationship is the single most common failure point in modern web audio applications.

11.1 The AudioWorklet Lifecycle

Implementing an AudioWorklet requires two distinct files due to the thread isolation model.

11.1.1 The Processor (Audio Thread)

The processor file (e.g., pitch-processor.js) extends the AudioWorkletProcessor class. This class must be registered with a unique name.

- **Constructor:** This is where memory allocation must happen. We initialize the Float32Array ring buffer here. Allocating memory inside the process method is strictly prohibited to avoid GC pauses.
- **Process Method:** The signature is process(inputs, outputs, parameters).
 - **inputs:** An array of inputs, each containing an array of channels. We usually care about inputs (First input, Left channel).
 - **Return Value:** The method must return true to keep the node alive. Returning

false (or undefined) allows the browser to garbage collect the node, silencing the audio processing.

Code Logic Narrative:

Inside the process method, the loop iterates through the 128 incoming samples. Each sample is copied to the pre-allocated ring buffer. A separate counter tracks how many samples have been added since the last detection run. When this counter exceeds the hopSize (e.g., 512 samples), the pitch detection logic is triggered.

The detection logic (imported from pitchy or implemented manually) scans the ring buffer. It calculates the pitch and a "clarity" metric. These two values are then packed into a message and sent via this.port.postMessage().

11.1.2 The Node (Main Thread)

The main thread script acts as the controller.

1. **Loading:** It calls `audioContext.audioWorklet.addModule('pitch-processor.js')`. This is an asynchronous network request.
2. **Instantiation:** Once the module is loaded, the node is created: `const node = new AudioWorkletNode(context, 'pitch-processor')`.
3. **Connection:** The microphone source is connected to this node: `micSource.connect(node)`.
4. **Output:** Crucially, the node must be connected to `context.destination` (the speakers) or the browser will think the node is unused and garbage collect it. However, connecting a microphone to speakers causes feedback loops.
 - **Mitigation:** Connect the node to a `GainNode` with `gain.value = 0`, and connect *that* to the destination. This keeps the processing graph "active" without producing audible feedback.

11.2 Handling High Sample Rates

Modern audio interfaces and some mobile devices operate at 48kHz, 96kHz, or even 192kHz.

- **Computational Load:** Running YIN at 96kHz requires processing twice as much data as 48kHz for the same time window.
- **Downsampling:** It is highly recommended to implement a simple Decimation filter (downsampler) inside the worklet. If the context sample rate is > 48kHz, averaging every 2 samples and throwing away the rest effectively halves the sample rate.
 - *Nyquist Implication**: Downsampling to 24kHz limits the maximum detectable frequency to 12kHz. Since the highest note on a piano (C8) is ~4186 Hz, this is perfectly acceptable for musical pitch detection and saves 50% of the CPU budget.

12. Visualization Mechanics: The "Ticker Tape"

The visualization requirement—a graph that scrolls as new notes are detected—can be implemented using two distinct strategies in the Canvas API. Understanding the difference is key to performance.

12.1 Strategy A: The "Shift and Draw" (Raster Manipulation)

This strategy treats the canvas as a raster image.

- **Mechanism:**
 1. Capture the entire canvas content as an image data block.
 2. Draw it back onto the canvas, shifted left by X pixels.
 3. Draw the new data in the gap created on the right.
- **Pros:** Constant complexity $O(1)$. The cost of drawing does not increase as the app runs for hours.
- **Cons:** "Smearing." Repeatedly resampling the image (reading and writing pixels) introduces anti-aliasing artifacts. Sharp lines become blurry blobs after scrolling across the screen. This can be mitigated by disabling image smoothing (`ctx.imageSmoothingEnabled = false`), but that results in jagged, pixelated lines.

12.2 Strategy B: The "Circular Buffer Path" (Vector Redraw)

This strategy treats the data as a vector path.

- **Mechanism:**
 1. Maintain a JavaScript array of the last N data points (where N corresponds to the screen width).
 2. Every frame, `ctx.clearRect()` the entire screen.
 3. `ctx.beginPath()`.
 4. Iterate through the array, calling `ctx.lineTo(x, y)`.
 5. `ctx.stroke()`.
- **Pros:** Crisp, perfect vector lines every frame. No degradation over time.
- **Cons:** Complexity is $O(N)$.
- **Analysis:** For a 1920px wide screen, $N \approx 2000$. Iterating 2000 items in a simple loop is trivial for modern JavaScript engines (taking microseconds). The browser's rasterizer can easily handle stroking a 2000-segment path at 60fps.
- **Verdict:** Strategy B is superior for visual quality and is the recommended approach for this project.

12.3 Handling High-DPI (Retina) Displays

Web Canvas appears blurry on Retina/High-DPI screens if not scaled correctly.

- **The Problem:** A canvas with `width="500"` has 500 logical pixels. On a 2x Retina screen, it is stretched to cover 1000 physical pixels, causing blur.
- **The Solution:**
JavaScript
`const dpr = window.devicePixelRatio |`

`| 1;`

`// Set internal memory size to physical pixels`

```

canvas.width = rect.width * dpr;
canvas.height = rect.height * dpr;
// Scale all drawing operations up
ctx.scale(dpr, dpr);
// Force CSS size to logical pixels
canvas.style.width = `${rect.width}px;
canvas.style.height = `${rect.height}px;
...

```

This ensures that the text and lines are rendered at the native resolution of the device, which is critical for a "professional" feel.²⁵

13. Advanced Algorithmic Considerations

13.1 The "Missing Fundamental" Problem

A common issue when detecting pitch from mobile phone speakers or cheap recordings is the "Missing Fundamental." A low note (e.g., 100Hz) might have very little energy at 100Hz due to the physical limitations of the speaker, but strong energy at 200Hz, 300Hz, etc.

- **Autocorrelation Failure:** Simple ACF often locks onto 200Hz (the strongest peak).
- **YIN/MPM Resilience:** These algorithms are designed to find the *periodicity* of the envelope, not just the spectral peak. They can often correctly infer the 100Hz fundamental even if the 100Hz component is missing, because the 200/300/400Hz harmonics align in phase every 10ms (100Hz period).
- **Implementation Tip:** If the app struggles with low notes, implementing a **spectral whitening** pre-processing step (flattening the frequency spectrum) can help, but this adds complexity. Generally, YIN handles this well enough out of the box.

13.2 Noise Gating and Voicing Detection

Pitch detectors run constantly, even when there is silence. This produces random output (garbage data) driven by the noise floor.

- **Clarity Metric:** Both YIN and MPM output a "clarity" (or "confidence") value between 0 and 1.
- **Thresholding:**
 - **Instruments:** A threshold of 0.9 is often used for clear, sustained tones.
 - **Voice:** A threshold of 0.6–0.7 is more appropriate, as breathy vocals are less periodic than a violin string.
- **Dynamic Thresholding:** An advanced implementation might lower the threshold dynamically if the volume (RMS amplitude) is high, assuming that loud sounds are likely intentional notes even if they are slightly noisy.

14. Comparative Analysis of Open Source Libraries

To assist in the "Technology Stack Options" requirement, we provide a detailed comparison of available libraries identified in the research.

Feature	Pitchy	AubioJS	Pitchfinder	ml5.js (CREPE)
Algorithm	McLeod Pitch Method (MPM)	YIN / YIN-FFT	Multiple (YIN, AMDF, MPM)	CNN (Deep Learning)
Language	TypeScript / JS	C++ (WASM)	JavaScript	TensorFlow.js
Size	Very Small (<10KB)	Large (~1MB WASM)	Small	Huge (Model weights)
Latency	Low (<20ms)	Ultra-Low (<10ms)	Low (<20ms)	High (>100ms)
Accuracy	High	High	Varies by algo	Very High (Noise robust)
Worklet Ready	Yes (Copy-paste friendly)	Yes (Requires loader)	Yes	No (Main thread only)
Best For	MVP / Web Apps	Pro Audio / DSP	Research / Testing	Offline / Transcription

Conclusion: **Pitchy** is the clear winner for a "Technical Spike" or MVP. It provides the right algorithm (MPM) in the right format (ES Modules) with minimal overhead. **AubioJS** is the upgrade path if performance issues arise on low-end hardware.

15. Final Roadmap and Recommendations

15.1 Phase 1: The Walking Skeleton

- **Objective:** Get a sine wave pitch detecting in the console.
- **Steps:**
 1. Set up a Vite project (Vanilla TS).
 2. Implement getUserMedia with echoCancellation: false.
 3. Create an AudioWorkletProcessor that simply logs "Processing" to the console.
 4. Import pitchy into the processor.
 5. Log the detected pitch of a 440Hz oscillator (generated by Web Audio) to verify accuracy.

15.2 Phase 2: The Visualization Loop

- **Objective:** Visualize the data.
- **Steps:**
 1. Create the MessagePort link.
 2. Build the Circular Buffer Rendering engine on Canvas.
 3. Map Frequency (Hz) to Y-Axis using the Logarithmic scale (critical for music, as linear Hz mapping makes high notes bunched together).
 4. Implement requestAnimationFrame to decouple rendering from the message rate.

15.3 Phase 3: Polish

- **Objective:** Make it feel "Real-Time."

- **Steps:**

1. Add a Median Filter (size 5) to the main thread data stream to kill jitter.
2. Implement the Note Name display ($C\#4 + 20$ cents).
3. Test on Mobile Safari to ensure the resume() context logic works.

This report confirms that the project is not only feasible but that the tools exist to create a highly performant, professional-quality application using standard web technologies. The key is architectural discipline: respect the threads, manage the memory, and trust the math of YIN/MPM.

Works cited

1. Web Audio API - MDN Web Docs - Mozilla, accessed December 18, 2025, https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
2. Detecting pitch with the Web Audio API and autocorrelation ..., accessed December 18, 2025, <https://alexanderell.is/posts/tuner/>
3. Audio Worklet is now available by default | Blog - Chrome for Developers, accessed December 18, 2025, <https://developer.chrome.com/blog/audio-worklet>
4. Background audio processing using AudioWorklet - Web APIs | MDN, accessed December 18, 2025, https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Using_Audio_Worklet
5. Implementing an AudioWorklet in Safari - Stack Overflow, accessed December 18, 2025, <https://stackoverflow.com/questions/73164697/implementing-an-audioworklet-in-safari>
6. Audio Worklet Support (10.2, Webpack 5) · vercel next.js · Discussion #62376 - GitHub, accessed December 18, 2025, <https://github.com/vercel/next.js/discussions/62376>
7. [audioworklet] Expose fetch() in AudioWorkletGlobalScope. · Issue #1439 · WebAudio/web-audio-api - GitHub, accessed December 18, 2025, <https://github.com/WebAudio/web-audio-api/issues/1439>
8. Pitch detection algorithm - Wikipedia, accessed December 18, 2025, https://en.wikipedia.org/wiki/Pitch_detection_algorithm
9. Pitch detection via auto correlation fails on higher pitches - Stack Overflow, accessed December 18, 2025, <https://stackoverflow.com/questions/22571227/pitch-detection-via-auto-correlation-fails-on-higher-pitches>
10. YIN, a fundamental frequency estimator for speech and music - Audition, accessed December 18, 2025, http://audition.ens.fr/adc/pdf/2002_JASA_YIN.pdf
11. pitchy - NPM, accessed December 18, 2025, <https://www.npmjs.com/package/pitchy>
12. pitch-detection/misc/mcleod/README.md at master - GitHub, accessed December 18, 2025, <https://github.com/sevagh/pitch-detection/blob/master/misc/mcleod/README.md>
13. (PDF) A smarter way to find pitch - ResearchGate, accessed December 18, 2025,

- https://www.researchgate.net/publication/230554927_A_smarter_way_to_find_pitch
14. CREPE: A Convolutional REpresentation for Pitch Estimation -- pre-trained model (ICASSP 2018) - GitHub, accessed December 18, 2025, <https://github.com/marl/crepe>
 15. [1802.06182] CREPE: A Convolutional Representation for Pitch Estimation - arXiv, accessed December 18, 2025, <https://arxiv.org/abs/1802.06182>
 16. CREPE: A Convolutional Representation for Pitch Estimation - ICASSP 2018 Lecture Session AASP-L4.3 - IEEE SigPort, accessed December 18, 2025, <https://sigport.org/sites/default/files/docs/crepe.pdf>
 17. CREPE Interactive Demo, accessed December 18, 2025, <https://marl.github.io/crepe/>
 18. Pitch Detection Implementation - General JUCE discussion, accessed December 18, 2025, <https://forum.juce.com/t/pitch-detection-implementation/66115>
 19. Comparing Rendering Performance of Common Web Technologies for Large Graphs - Interactive Media Lab Dresden, accessed December 18, 2025, <https://imld.de/cnt/uploads/Horak-2018-Graph-Performance.pdf>
 20. Real time data plotting performance HTML5 canvas vs Dom appending - Stack Overflow, accessed December 18, 2025, <https://stackoverflow.com/questions/24019174/real-time-data-plotting-performance-html5-canvas-vs-dom-appending>
 21. WebGL vs Canvas: Best Choice for Browser-Based CAD Tools | by AlterSquare - Medium, accessed December 18, 2025, <https://altersquare.medium.com/webgl-vs-canvas-best-choice-for-browser-based-cad-tools-231097daf063>
 22. HTML Canvas vs WebGL for whiteboarding app? - Reddit, accessed December 18, 2025, https://www.reddit.com/r/webgl/comments/1ftkfxl/html_canvas_vs_webgl_for_whiteboarding_app/
 23. Optimising HTML5 Canvas Rendering: Best Practices and Techniques - AG Grid Blog, accessed December 18, 2025, <https://blog.ag-grid.com/optimising-html5-canvas-rendering-best-practices-and-techniques/>
 24. large HTML5 Canvas scrolling performance tips - Stack Overflow, accessed December 18, 2025, <https://stackoverflow.com/questions/27613314/large-html5-canvas-scrolling-performance-tips>
 25. Optimizing canvas - Web APIs | MDN, accessed December 18, 2025, https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas
 26. cwilso/PitchDetect: Pitch detection in Web Audio using autocorrelation - GitHub, accessed December 18, 2025, <https://github.com/cwilso/PitchDetect>
 27. Command line tools — aubio 0.4.9 documentation, accessed December 18, 2025, <https://aubio.org/manual/latest/cli.html>
 28. How to achieve top-notch scrolling performance using HTML5 Canvas - DEV

Community, accessed December 18, 2025,

<https://dev.to/sip3/how-to-achieve-top-notch-scrolling-performance-using-html5-canvas-k49>

29. peterkhayes/pitchfinder: A compilation of pitch detection algorithms for Javascript. - GitHub, accessed December 18, 2025,

<https://github.com/peterkhayes/pitchfinder>